



UITs

Future will be better than the past

University of Information Technology & Sciences

An initiative of **PHP** Family

Microprocessors, Microcontrollers and Assembly Language Lab

Lab Report - 2

Course Title : Microprocessors, Microcontrollers and Assembly
Language Lab

Course Code : CSE0611324

Submitted to:

Teacher's Name	Designation
Tahmina Yeasmin Lima	Lecturer
Abrar Mahbub Tanim	Lecturer

Submitted by:

Student Name : Gaus Saraf Murady

Student ID : 0432410005101088

Semester : Autumn 2026

Batch : 55

Department : CSE

Date of Submission: 16.08.2026

Experiment No: 2

Experiment Name: Arithmetic operation of two one digit values in assembly language

Introduction:

In this lab, we learnt about programming with Assembly Language by performing some arithmetic operations. We operated on 2 single digit inputs and performed Addition, Subtraction, Multiplication, and Division using them.

Code:

```
.model small
.stack 100h

.data
    num1 db ?
    num2 db ?
    sum db ?
    subt db ?
    multi db ?
    divi db ?

    quot db ?
    rem db ?

.code

main proc
    mov ax, @data
    mov ds, ax

    mov ah, 1
    int 21h
    sub al, 48 ; subtracting to
convert from ASCII to
Number values for operation
    mov num1, al ; similar to
storing to `bh`

; //////////////////////////////////
mov ah, 2
mov dl, 32
int 21h
; //////////////////////////////////

mov ah, 1
int 21h
sub al, 48
mov num2, al

; //////////////////////////////////
mov ah, 2
mov dl, 32
int 21h
; //////////////////////////////////

; SUM
```

```
    mov al, num1 ; can use
`ADD num1, num2` but we
will lose the value of `num1`
    add al, num2
    add al, 48
    mov sum, al ; summation
for converting Number
value back to ASCII for
Output

    mov ah, 2
    mov dl, sum
    int 21h

; //////////////////////////////////
mov ah, 2
mov dl, 32
int 21h
; //////////////////////////////////

; SUB

    mov al, num1
    sub al, num2
    add al, 48
    mov subt, al

    mov ah, 2
    mov dl, subt
    int 21h

; //////////////////////////////////
mov ah, 2
mov dl, 32
int 21h
; //////////////////////////////////

; MUL

    mov al, num1
    mul num2 ; AL is
automatically multiplied by
num2
    add al, 48
    mov multi, al

; // ALT
; mov al, num1
; mov bl, num2
; mul bl
; mov multi, al
```

```
; add multi, 30h

    mov ah, 2
    mov dl, multi
    int 21h

; //////////////////////////////////
mov ah, 2
mov dl, 32
int 21h
; //////////////////////////////////

; DIV

    mov al, num1
    div num2
    add al, 48
    add ah, 48
    mov divi, al
    mov rem, ah

    mov ah, 2
    mov dl, divi
    int 21h

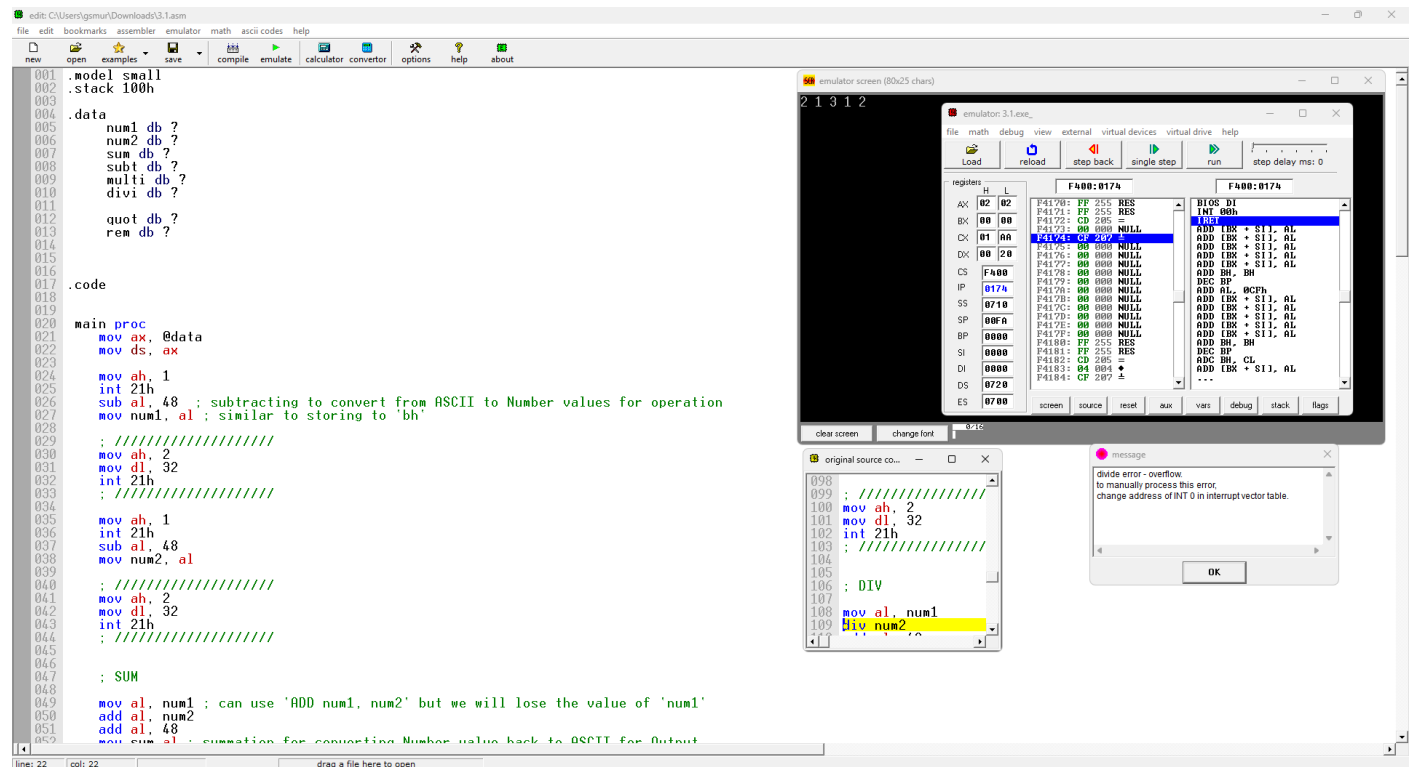
; //////////////////////////////////
mov ah, 2
mov dl, 32
int 21h
; //////////////////////////////////

    mov ah, 2
    mov dl, rem
    int 21h

; // ALT
; mov al, num1
; mov bl, num2
; div bl
; mov quot, al
; mov rem, ah
; add quot, 30h
; add rem, 30h

    main endp
end main
```

Output:



```
001 .model small
002 .stack 100h
003
004 .data
005     num1 db ?
006     num2 db ?
007     sum db ?
008     subtr db ?
009     multi db ?
010     divi db ?
011
012     quot db ?
013     rem db ?
014
015
016
017
018 .code
019
020 main proc
021     mov ax, @data
022     mov ds, ax
023
024     mov ah, 1
025     int 21h
026     sub al, 48 ; subtracting to convert from ASCII to Number values for operation
027     mov num1, al ; similar to storing to 'bh'
028
029     ; //////////////////////////////////
030     mov ah, 2
031     mov dl, 32
032     int 21h
033     ; //////////////////////////////////
034
035     mov ah, 1
036     int 21h
037     sub al, 48
038     mov num2, al
039
040     ; //////////////////////////////////
041     mov ah, 2
042     mov dl, 32
043     int 21h
044     ; //////////////////////////////////
045
046
047     ; SUM
048
049     mov al, num1 ; can use 'ADD num1, num2' but we will lose the value of 'num1'
050     add al, num2
051     mov sum, al ; summation for converting Number value back to ASCII for Output
052
```

Conclusion:

We successfully showed the expected outputs of the respective operations. The MUL and DIV operations were a bit special since they require only a single parameter to operate. Also, the handling of double digit output of the operations using DIV was interesting, even though we could not implement it correctly during the lab hours.